How Good Are Query Optimizers, Really?

Viktor Leis
TUM
leis@in.tum.de

Andrey Gubichev
TUM
gubichev@in.tum.de

Atanas Mirchev
TUM
mirchev@in.tum.de

Peter Boncz
CWI
p.boncz@cwi.nl

Alfons Kemper
TUM
kemper@in.tum.de

Thomas Neumann
TUM
neumann@in.tum.de

ABSTRACT

Finding a good join order is crucial for query performance. In this paper, we introduce the Join Order Benchmark (JOB) and experimentally revisit the main components in the classic query optimizer architecture using a complex, real-world data set and realistic multi-join queries. We investigate the quality of industrial-strength cardinality estimators and find that all estimators routinely produce large errors. We further show that while estimates are essential for finding a good join order, query performance is unsatisfactory if the query engine relies too heavily on these estimates. Using another set of experiments that measure the impact of the cost model, we find that it has much less influence on query performance than the cardinality estimates. Finally, we investigate plan enumeration techniques comparing exhaustive dynamic programming with heuristic algorithms and find that exhaustive enumeration improves performance despite the sub-optimal cardinality estimates.

1. INTRODUCTION

The problem of finding a good join order is one of the most studied problems in the database field. Figure 1 illustrates the classical, cost-based approach, which dates back to System R [36]. To obtain an efficient query plan, the query optimizer enumerates some subset of the valid join orders, for example using dynamic programming. Using cardinality estimates as its principal input, the cost model then chooses the cheapest alternative from semantically equivalent plan alternatives.

Theoretically, as long as the cardinality estimations and the cost model are accurate, this architecture obtains the optimal query plan. In reality, cardinality estimates are usually computed based on simplifying assumptions like uniformity and independence. In real-world data sets, these assumptions are *frequently* wrong, which may lead to sub-optimal and sometimes disastrous plans.

In this *experiments and analyses* paper we investigate the three main components of the classical query optimization architecture in order to answer the following questions:

- How good are cardinality estimators and when do bad estimates lead to slow queries?

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-nc-nd/4.0/. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org.
Proceedings of the VLDB Endowment, Vol. 9, No. 3
Copyright 2015 VLDB Endowment 2150-8097/15/11.

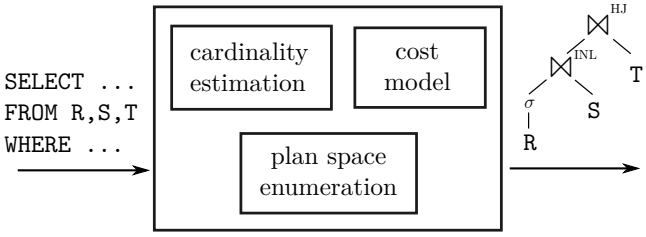


Figure 1: Traditional query optimizer architecture

- How important is an accurate cost model for the overall query optimization process?
- How large does the enumerated plan space need to be?

To answer these questions, we use a novel methodology that allows us to isolate the influence of the individual optimizer components on query performance. Our experiments are conducted using a real-world data set and 113 multi-join queries that provide a challenging, diverse, and realistic workload. Another novel aspect of this paper is that it focuses on the increasingly common main-memory scenario, where all data fits into RAM.

The main contributions of this paper are listed in the following:

- We design a challenging workload named *Join Order Benchmark (*JOB*)*, which is based on the IMDB data set. The benchmark is publicly available to facilitate further research.
- To the best of our knowledge, this paper presents the first end-to-end study of the join ordering problem using a real-world data set and realistic queries.
- By quantifying the contributions of cardinality estimation, the cost model, and the plan enumeration algorithm on query performance, we provide guidelines for the complete design of a query optimizer. We also show that many disastrous plans can easily be avoided.

The rest of this paper is organized as follows: We first discuss important background and our new benchmark in Section 2. Section 3 shows that the cardinality estimators of the major relational database systems produce bad estimates for many realistic queries, in particular for multi-join queries. The conditions under which these bad estimates cause slow performance are analyzed in Section 4. We show that it very much depends on how much the query engine relies on these estimates and on how complex the physical database design is, i.e., the number of indexes available. Query engines that mainly rely on hash joins and full table scans,

are quite robust even in the presence of large cardinality estimation errors. The more indexes are available, the harder the problem becomes for the query optimizer resulting in runtimes that are far away from the optimal query plan. Section 5 shows that with the currently-used cardinality estimation techniques, the influence of cost model errors is dwarfed by cardinality estimation errors and that even quite simple cost models seem to be sufficient. Section 6 investigates different plan enumeration algorithms and shows that—despite large cardinality misestimates and sub-optimal cost models—exhaustive join order enumeration improves performance and that using heuristics leaves performance on the table. Finally, after discussing related work in Section 7, we present our conclusions and future work in Section 8.

2. BACKGROUND AND METHODOLOGY

Many query optimization papers ignore cardinality estimation and only study search space exploration for join ordering with randomly generated, synthetic queries (e.g., [32, 13]). Other papers investigate only cardinality estimation in isolation either theoretically (e.g., [21]) or empirically (e.g., [43]). As important and interesting both approaches are for understanding query optimizers, they do not necessarily reflect real-world user experience.

The goal of this paper is to investigate the contribution of all relevant query optimizer components to end-to-end query performance in a realistic setting. We therefore perform our experiments using a workload based on a real-world data set and the widely-used PostgreSQL system. PostgreSQL is a relational database system with a fairly traditional architecture making it a good subject for our experiments. Furthermore, its open source nature allows one to inspect and change its internals. In this section we introduce the Join Order Benchmark, describe all relevant aspects of PostgreSQL, and present our methodology.

2.1 The IMDB Data Set

Many research papers on query processing and optimization use standard benchmarks like TPC-H, TPC-DS, or the Star Schema Benchmark (SSB). While these benchmarks have proven their value for evaluating query engines, we argue that they are not good benchmarks for the cardinality estimation component of query optimizers. The reason is that in order to easily be able to scale the benchmark data, the data generators are using the very same simplifying assumptions (uniformity, independence, principle of inclusion) that query optimizers make. Real-world data sets, in contrast, are full of correlations and non-uniform data distributions, which makes cardinality estimation much harder. Section 3.3 shows that PostgreSQL's simple cardinality estimator indeed works unrealistically well for TPC-H.

Therefore, instead of using a synthetic data set, we chose the *Internet Movie Data Base*[1] *(IMDB)*. It contains a plethora of information about movies and related facts about actors, directors, production companies, etc. The data is freely available[2] for non-commercial use as text files. In addition, we used the open-source *imdbpy*[3] package to transform the text files into a relational database with 21 tables. The data set allows one to answer queries like "Which actors played in movies released between 2000 and 2005 with ratings above 8?". Like most real-world data sets IMDB is full of correlations and non-uniform data distributions, and is therefore much more challenging than most synthetic data sets. Our snapshot is from May 2013 and occupies 3.6 GB when exported to CSV files. The two largest tables, `cast_info` and `movie_info` have 36 M and 15 M rows, respectively.

[1] `http://www.imdb.com/`

[2] `ftp://ftp.fu-berlin.de/pub/misc/movies/database/`

[3] `https://bitbucket.org/alberanid/imdbpy/get/5.0.zip`

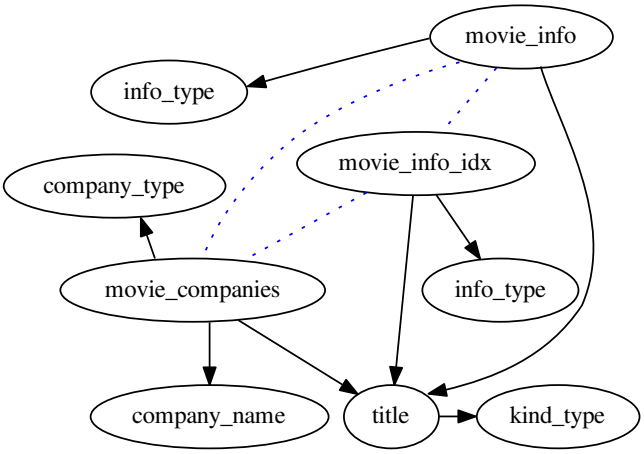


Figure 2: Typical query graph of our workload

2.2 The JOB Queries

Based on the IMDB database, we have constructed analytical SQL queries. Since we focus on join ordering, which arguably is the most important query optimization problem, we designed the queries to have between 3 and 16 joins, with an average of 8 joins per query. Query 13d, which finds the ratings and release dates for all movies produced by US companies, is a typical example:

```
SELECT cn.name, mi.info, miidx.info
FROM company_name cn, company_type ct,
     info_type it, info_type it2, title t,
     kind_type kt, movie_companies mc,
     movie_info mi, movie_info_idx miidx
WHERE cn.country_code ='[us]'
AND ct.kind = 'production companies'
AND it.info = 'rating'
AND it2.info = 'release dates'
AND kt.kind = 'movie'
AND ... -- (11 join predicates)
```

Each query consists of one select-project-join block[4]. The join graph of the query is shown in Figure 2. The solid edges in the graph represent key/foreign key edges ($1 : n$) with the arrow head pointing to the primary key side. Dotted edges represent foreign key/foreign key joins ($n : m$), which appear due to transitive join predicates. Our query set consists of 33 query structures, each with 2-6 variants that differ in their selections only, resulting in a total of 113 queries. Note that depending on the selectivities of the base table predicates, the variants of the same query structure have different optimal query plans that yield widely differing (sometimes by orders of magnitude) runtimes. Also, some queries have more complex selection predicates than the example (e.g., disjunctions or substring search using `LIKE`).

Our queries are "realistic" and "ad hoc" in the sense that they answer questions that may reasonably have been asked by a movie

[4] Since in this paper we do not model or investigate aggregation, we omitted `GROUP BY` from our queries. To avoid communication from becoming the performance bottleneck for queries with large result sizes, we wrap all attributes in the projection clause with `MIN(...)` expressions when executing (but not when estimating). This change has no effect on PostgreSQL's join order selection because its optimizer does not push down aggregations.

enthusiast. We also believe that despite their simple SPJ-structure, the queries model the core difficulty of the join ordering problem. For cardinality estimators the queries are challenging due to the significant number of joins and the correlations contained in the data set. However, we did not try to "trick" the query optimizer, e.g., by picking attributes with extreme correlations. Also, we intentionally did not include more complex join predicates like inequalities or non-surrogate-key predicates, because cardinality estimation for this workload is already quite challenging.

We propose JOB for future research in cardinality estimation and query optimization. The query set is available online:
`http://www-db.in.tum.de/~leis/qo/job.tgz`

2.3 PostgreSQL

PostgreSQL's optimizer follows the traditional textbook architecture. Join orders, including bushy trees but excluding trees with cross products, are enumerated using dynamic programming. The cost model, which is used to decide which plan alternative is cheaper, is described in more detail in Section 5.1. The cardinalities of base tables are estimated using histograms (quantile statistics), most common values with their frequencies, and domain cardinalities (distinct value counts). These per-attribute statistics are computed by the `analyze` command using a sample of the relation. For complex predicates, where histograms can not be applied, the system resorts to ad hoc methods that are not theoretically grounded ("magic constants"). To combine conjunctive predicates for the same table, PostgreSQL simply assumes independence and multiplies the selectivities of the individual selectivity estimates.

The result sizes of joins are estimated using the formula

$$|T_1 \bowtie_{x=y} T_2| = \frac{|T_1||T_2|}{\max(\text{dom}(x), \text{dom}(y))},$$

where T_1 and T_2 are arbitrary expressions and $\text{dom}(x)$ is the domain cardinality of attribute x, i.e., the number of distinct values of x. This value is the principal input for the join cardinality estimation. To summarize, PostgreSQL's cardinality estimator is based on the following assumptions:

- uniformity: all values, except for the most-frequent ones, are assumed to have the same number of tuples
- independence: predicates on attributes (in the same table or from joined tables) are independent
- principle of inclusion: the domains of the join keys overlap such that the keys from the smaller domain have matches in the larger domain

The query engine of PostgreSQL takes a physical operator plan and executes it using Volcano-style interpretation. The most important access paths are full table scans and lookups in unclustered B+Tree indexes. Joins can be executed using either nested loops (with or without index lookups), in-memory hash joins, or sort-merge joins where the sort can spill to disk if necessary. The decision which join algorithm is used is made by the optimizer and cannot be changed at runtime.

2.4 Cardinality Extraction and Injection

We loaded the IMDB data set into 5 relational database systems: PostgreSQL, HyPer, and 3 commercial systems. Next, we ran the statistics gathering command of each database system with default settings to generate the database-specific statistics (e.g., histograms or samples) that are used by the estimation algorithms. We then obtained the cardinality estimates for all intermediate results of our test queries using database-specific commands (e.g., using the `EXPLAIN` command for PostgreSQL). We will later use these estimates of different systems to obtain optimal query plans (w.r.t. respective systems) and run these plans in PostgreSQL. For example, the intermediate results of the chain query

$$\sigma_{x=5}(A) \bowtie_{A.bid=B.id} B \bowtie_{B.cid=C.id} C$$

are $\sigma_{x=5}(A)$, $\sigma_{x=5}(A) \bowtie B$, $B \bowtie C$, and $\sigma_{x=5}(A) \bowtie B \bowtie C$. Additionally, the availability of indexes on foreign keys and index-nested loop joins introduces the need for additional intermediate result sizes. For instance, if there exists a non-unique index on the foreign key $A.bid$, it is also necessary to estimate $A \bowtie B$ and $A \bowtie B \bowtie C$. The reason is that the selection $A.x = 5$ can only be applied *after* retrieving all matching tuples from the index on $A.bid$, and therefore the system produces two intermediate results, before and after the selection. Besides cardinality estimates from the different systems, we also obtain the true cardinality for each intermediate result by executing `SELECT COUNT(*)` queries[5].

We further modified PostgreSQL to enable cardinality injection of arbitrary join expressions, allowing PostgreSQL's optimizer to use the estimates of other systems (or the true cardinality) instead of its own. This allows one to directly measure the influence of cardinality estimates from different systems on query performance. Note that IBM DB2 allows a limited form of user control over the estimation process by allowing users to explicitly specify the selectivities of predicates. However, selectivity injection cannot fully model inter-relation correlations and is therefore less general than the capability of injecting cardinalities for arbitrary expressions.

2.5 Experimental Setup

The cardinalities of the commercial systems were obtained using a laptop running Windows 7. All performance experiments were performed on a server with two Intel Xeon X5570 CPUs (2.9 GHz) and a total of 8 cores running PostgreSQL 9.4 on Linux. PostgreSQL does not parallelize queries, so that only a single core was used during query processing. The system has 64 GB of RAM, which means that the entire IMDB database is fully cached in RAM. Intermediate query processing results (e.g., hash tables) also easily fit into RAM, unless a very bad plan with extremely large intermediate results is chosen.

We set the memory limit per operator (`work_mem`) to 2 GB, which results in much better performance due to the more frequent use of in-memory hash joins instead of external memory sort-merge joins. Additionally, we set the buffer pool size (`shared_buffers`) to 4 GB and the size of the operating system's buffer cache used by PostgreSQL (`effective_cache_size`) to 32 GB. For PostgreSQL it is generally recommended to use OS buffering in addition to its own buffer pool and keep most of the memory on the OS side. The defaults for these three settings are very low (MBs, not GBs), which is why increasing them is generally recommended. Finally, by increasing the `geqo_threshold` parameter to 18 we forced PostgreSQL to always use dynamic programming instead of falling back to a heuristic for queries with more than 12 joins.

3. CARDINALITY ESTIMATION

Cardinality estimates are the most important ingredient for finding a good query plan. Even exhaustive join order enumeration and a perfectly accurate cost model are worthless unless the cardinality estimates are (roughly) correct. It is well known, however, that

[5]For our workload it was still feasible to do this naïvely. For larger data sets the approach by Chaudhuri et al. [7] may become necessary.

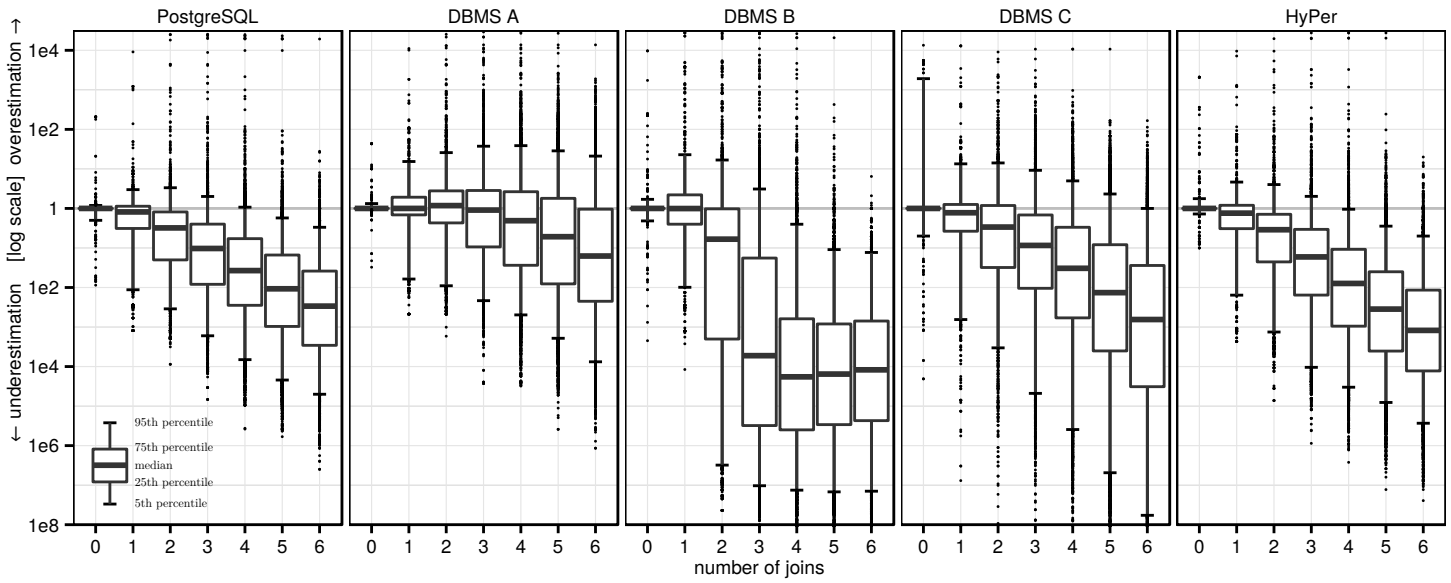


Figure 3: Quality of cardinality estimates for multi-join queries in comparison with the true cardinalities. Each boxplot summarizes the error distribution of all subexpressions with a particular size (over all queries in the workload)

	median	90th	95th	max
PostgreSQL	1.00	2.08	6.10	207
DBMS A	1.01	1.33	1.98	43.4
DBMS B	1.00	6.03	30.2	104000
DBMS C	1.06	1677	5367	20471
HyPer	1.02	4.47	8.00	2084

Table 1: Q-errors for base table selections

cardinality estimates are sometimes wrong by orders of magnitude, and that such errors are usually the reason for slow queries. In this section, we experimentally investigate the quality of cardinality estimates in relational database systems by comparing the estimates with the true cardinalities.

3.1 Estimates for Base Tables

To measure the quality of base table cardinality estimates, we use the *q-error*, which is the factor by which an estimate differs from the true cardinality. For example, if the true cardinality of an expression is 100, the estimates of 10 or 1000 both have a q-error of 10. Using the ratio instead of an absolute or quadratic difference captures the intuition that for making planning decisions only relative differences matter. The q-error furthermore provides a theoretical upper bound for the plan quality if the q-errors of a query are bounded [30].

Table 1 shows the 50th, 90th, 95th, and 100th percentiles of the q-errors for the 629 base table selections in our workload. The median q-error is close to the optimal value of 1 for all systems, indicating that the majority of all selections are estimated correctly. However, all systems produce misestimates for some queries, and the quality of the cardinality estimates differs strongly between the different systems.

Looking at the individual selections, we found that DBMS A and HyPer can usually predict even complex predicates like substring search using `LIKE` very well. To estimate the selectivities *for base tables* HyPer uses a random sample of 1000 rows per table and applies the predicates on that sample. This allows one to get accurate estimates for arbitrary base table predicates as long as the selectivity is not too low. When we looked at the selections where DBMS A and HyPer produce errors above 2, we found that most of them have predicates with extremely low true selectivities (e.g., 10^{-5} or 10^{-6}). This routinely happens when the selection yields zero tuples on the sample, and the system falls back on an ad-hoc estimation method ("magic constants"). It therefore appears to be likely that DBMS A also uses the sampling approach.

The estimates of the other systems are worse and seem to be based on per-attribute histograms, which do not work well for many predicates and cannot detect (anti-)correlations between attributes. Note that we obtained all estimates using the default settings after running the respective statistics gathering tool. Some commercial systems support the use of sampling for base table estimation, multi-attribute histograms ("column group statistics"), or ex post feedback from previous query runs [38]. However, these features are either not enabled by default or are not fully automatic.

3.2 Estimates for Joins

Let us now turn our attention to the estimation of intermediate results for joins, which are more challenging because sampling or histograms do not work well. Figure 3 summarizes over 100,000 cardinality estimates in a single figure. For each intermediate result of our query set, we compute the factor by which the estimate differs from the true cardinality, distinguishing between over- and underestimation. The graph shows one "boxplot" (note the legend in the bottom-left corner) for each intermediate result size, which allows one to compare how the errors change as the number of joins increases. The vertical axis uses a logarithmic scale to encompass underestimates by a factor of 10^8 and overestimates by a factor of 10^4.

Despite the better base table estimates of DBMS A, the overall variance of the join estimation errors, as indicated by the boxplot, is similar for all systems with the exception of DBMS B. For all systems we routinely observe misestimates by a factor of 1000 or more. Furthermore, as witnessed by the increasing height of the box plots, the errors grow exponentially (note the logarithmic scale)

as the number of joins increases [21]. For PostgreSQL 16% of the estimates for 1 join are wrong by a factor of 10 or more. This percentage increases to 32% with 2 joins, and to 52% with 3 joins. For DBMS A, which has the best estimator of the systems we compared, the corresponding percentages are only marginally better at 15%, 25%, and 36%.

Another striking observation is that all tested systems—though DBMS A to a lesser degree—tend to systematically underestimate the results sizes of queries with multiple joins. This can be deduced from the median of the error distributions in Figure 3. For our query set, it is indeed the case that the intermediate results tend to decrease with an increasing number of joins because more base table selections get applied. However, the true decrease is less than the independence assumption used by PostgreSQL (and apparently by the other systems) predicts. Underestimation is most pronounced with DBMS B, which frequently estimates 1 row for queries with more than 2 joins. The estimates of DBMS A, on the other hand, have medians that are much closer to the truth, despite their variance being similar to some of the other systems. We speculate that DBMS A uses a damping factor that depends on the join size, similar to how many optimizers combine multiple selectivities. Many estimators combine the selectivities of multiple predicates (e.g., for a base relation or for a subexpression with multiple joins) not by assuming full independence, but by adjusting the selectivities "upwards", using a damping factor. The motivation for this stems from the fact that the more predicates need to be applied, the less certain one should be about their independence.

Given the simplicity of PostgreSQL's join estimation formula (cf. Section 2.3) and the fact that its estimates are nevertheless competitive with the commercial systems, we can deduce that the current join size estimators are based on the independence assumption. No system tested was able to detect join-crossing correlations. Furthermore, cardinality estimation is highly brittle, as illustrated by the significant number of extremely large errors we observed (factor 1000 or more) and the following anecdote: In PostgreSQL, we observed different cardinality estimates of the same simple 2-join query depending on the *syntactic* order of the relations in the `from` and/or the join predicates in the `where` clauses! Simply by swapping predicates or relations, we observed the estimates of 3, 9, 128, or 310 rows for the same query (with a true cardinality of 2600)[6].

Note that this section does not benchmark the query optimizers of the different systems. In particular, our results do not imply that the DBMS B's optimizer or the resulting query performance is necessarily worse than that of other systems, despite larger errors in the estimator. The query runtime heavily depends on how the system's optimizer uses the estimates and how much trust it puts into these numbers. A sophisticated engine may employ adaptive operators (e.g., [4, 8]) and thus mitigate the impact of misestimations. The results do, however, demonstrate that the state-of-the-art in cardinality estimation is far from perfect.

3.3 Estimates for TPC-H

We have stated earlier that cardinality estimation in TPC-H is a rather trivial task. Figure 4 substantiates that claim by showing the distributions of PostgreSQL estimation errors for 3 of the larger TPC-H queries and 4 of our JOB queries. Note that in the figure we report estimation errors for *individual* queries (not for all queries like in Figure 3). Clearly, the TPC-H query workload does not present many hard challenges for cardinality estimators. In contrast, our workload contains queries that routinely lead to severe overestimation and underestimation errors, and hence can be considered a challenging benchmark for cardinality estimation.

[6] The reasons for this surprising behavior are two implementation artifacts: First, estimates that are less than 1 are rounded up to 1, making subexpression estimates sensitive to the (usually arbitrary) join enumeration order, which is affected by the `from` clause. The second is a consistency problem caused by incorrect domain sizes of predicate attributes in joins with multiple predicates.

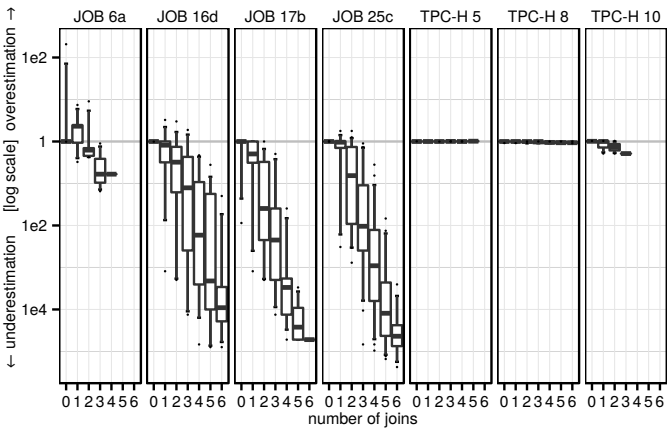


Figure 4: PostgreSQL cardinality estimates for 4 JOB queries and 3 TPC-H queries

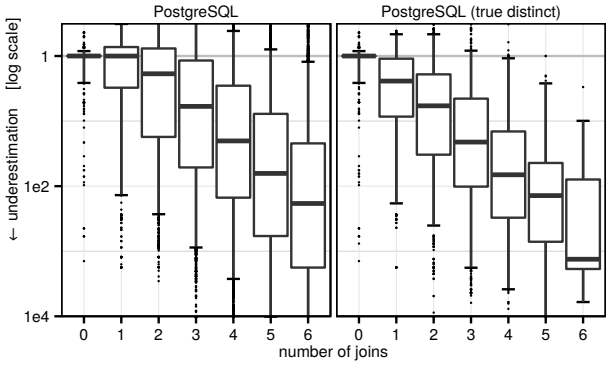


Figure 5: PostgreSQL cardinality estimates based on the default distinct count estimates, and the true distinct counts

3.4 Better Statistics for PostgreSQL

As mentioned in Section 2.3, the most important statistic for join estimation in PostgreSQL is the number of distinct values. These statistics are estimated from a fixed-sized sample, and we have observed severe underestimates for large tables. To determine if the misestimated distinct counts are the underlying problem for cardinality estimation, we computed these values precisely and replaced the estimated with the true values.

Figure 5 shows that the true distinct counts slightly improve the variance of the errors. Surprisingly, however, the trend to underestimate cardinalities becomes even more pronounced. The reason is that the original, underestimated distinct counts resulted in higher estimates, which, accidentally, are closer to the truth. This is an example for the proverbial "two wrongs that make a right", i.e., two errors that (partially) cancel each other out. Such behavior makes analyzing and fixing query optimizer problems very frustrating because fixing one query might break another.

4. WHEN DO BAD CARDINALITY ESTIMATES LEAD TO SLOW QUERIES?

While the large estimation errors shown in the previous section are certainly sobering, large errors do *not necessarily* lead to slow query plans. For example, the misestimated expression may be cheap in comparison with other parts of the query, or the relevant plan alternative may have been misestimated by a similar factor thus "canceling out" the original error. In this section we investigate the conditions under which bad cardinalities are likely to cause slow queries.

One important observation is that query optimization is closely intertwined with the physical database design: the type and number of indexes heavily influence the plan search space, and therefore affects how sensitive the system is to cardinality misestimates. We therefore start this section with experiments using a relatively robust physical design with only primary key indexes and show that in such a setup the impact of cardinality misestimates can largely be mitigated. After that, we demonstrate that for more complex configurations with many indexes, cardinality misestimation makes it much more likely to miss the optimal plan by a large margin.

4.1 The Risk of Relying on Estimates

To measure the impact of cardinality misestimation on query performance we injected the estimates of the different systems into PostgreSQL and then executed the resulting plans. Using the same query engine allows one to compare the cardinality estimation components in isolation by (largely) abstracting away from the different query execution engines. Additionally, we inject the true cardinalities, which computes the—with respect to the cost model—optimal plan. We group the runtimes based on their slowdown w.r.t. the optimal plan, and report the distribution in the following table, where each column corresponds to a group:

	<0.9	[0.9,1.1)	[1.1,2)	[2,10)	[10,100)	>100
PostgreSQL	1.8%	38%	25%	25%	5.3%	5.3%
DBMS A	2.7%	54%	21%	14%	0.9%	7.1%
DBMS B	0.9%	35%	18%	15%	7.1%	25%
DBMS C	1.8%	38%	35%	13%	7.1%	5.3%
HyPer	2.7%	37%	27%	19%	8.0%	6.2%

A small number of queries become slightly slower using the true instead of the erroneous cardinalities. This effect is caused by cost model errors, which we discuss in Section 5. However, as expected, the vast majority of the queries are slower when estimates are used. Using DBMS A's estimates, 78% of the queries are less than 2× slower than using the true cardinalities, while for DBMS B this is the case for only 53% of the queries. This corroborates the findings about the relative quality of cardinality estimates in the previous section. Unfortunately, all estimators occasionally lead to plans that take an unreasonable time and lead to a timeout. Surprisingly, however, many of the observed slowdowns are easily avoidable despite the bad estimates as we show in the following.

When looking at the queries that did not finish in a reasonable time using the estimates, we found that most have one thing in common: PostgreSQL's optimizer decides to introduce a nested-loop join (without an index lookup) because of a very low cardinality estimate, whereas in reality the true cardinality is larger. As we saw in the previous section, systematic underestimation happens very frequently, which occasionally results in the introduction of nested-loop joins.

The underlying reason why PostgreSQL chooses nested-loop joins is that it picks the join algorithm on a purely cost-based basis. For example, if the cost estimate is 1,000,000 with the nested-loop join algorithm and 1,000,001 with a hash join, PostgreSQL will always prefer the nested-loop algorithm even if there is a equality join predicate, which allows one to use hashing. Of course, given the $O(n^2)$ complexity of nested-loop join and $O(n)$ complexity of hash join, and given the fact that underestimates are quite frequent, this decision is extremely risky. And even if the estimates happen to be correct, any potential performance advantage of a nested-loop join in comparison with a hash join is very small, so taking this *high risk* can only result in a *very small payoff*.

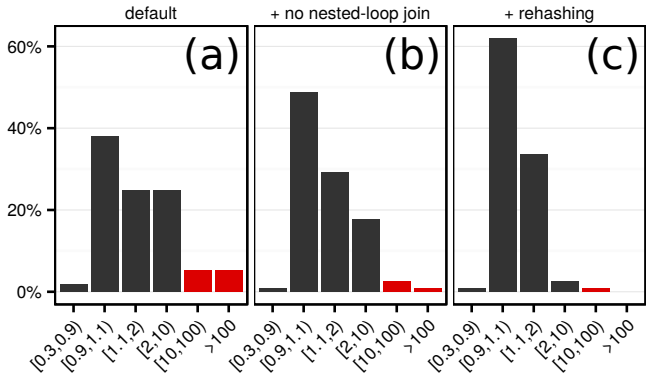


Figure 6: Slowdown of queries using PostgreSQL estimates w.r.t. using true cardinalities (primary key indexes only)

Therefore, we disabled nested-loop joins (but not index-nested-loop joins) in all following experiments. As Figure 6b shows, when rerunning all queries without these risky nested-loop joins, we observed no more timeouts despite using PostgreSQL's estimates.

Also, none of the queries performed slower than before despite having less join algorithm options, confirming our hypothesis that nested-loop joins (without indexes) seldom have any upside. However, this change does not solve all problems, as there are still a number of queries that are more than a factor of 10 slower (cf., red bars) in comparison with the true cardinalities.

When investigating the reason why the remaining queries still did not perform as well as they could, we found that most of them contain a hash join where the size of the build input is underestimated. PostgreSQL up to and including version 9.4 chooses the size of the in-memory hash table based on the cardinality estimate. Underestimates can lead to undersized hash tables with very long collisions chains and therefore bad performance. The upcoming version 9.5 resizes the hash table at runtime based on the number of rows actually stored in the hash table. We backported this patch to our code base, which is based on 9.4, and enabled it for all remaining experiments. Figure 6c shows the effect of this change in addition with disabled nested-loop joins. Less than 4% of the queries are off by more than 2× in comparison with the true cardinalities.

To summarize, being "purely cost-based", i.e., not taking into account the inherent uncertainty of cardinality estimates and the asymptotic complexities of different algorithm choices, can lead to very bad query plans. Algorithms that seldom offer a large benefit over more robust algorithms should not be chosen. Furthermore, query processing algorithms should, if possible, automatically determine their parameters at runtime instead of relying on cardinality estimates.

4.2 Good Plans Despite Bad Cardinalities

The query runtimes of plans with different join orders often vary by many orders of magnitude (cf. Section 6.1). Nevertheless, when

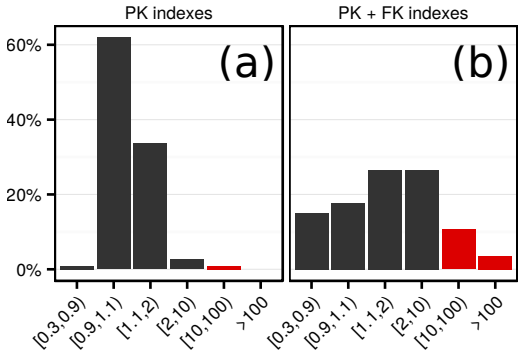


Figure 7: Slowdown of queries using PostgreSQL estimates w.r.t. using true cardinalities (different index configurations)

the database has only primary key indexes, as in all in experiments so far, and once nested loop joins have been disabled and rehashing has been enabled, the performance of most queries is close to the one obtained using the true cardinalities. Given the bad quality of the cardinality estimates, we consider this to be a surprisingly positive result. It is worthwhile to reflect on why this is the case.

The main reason is that without foreign key indexes, most large ("fact") tables need to be scanned using full table scans, which dampens the effect of different join orders. The join order still matters, but the results indicate that the cardinality estimates are usually good enough to rule out all disastrous join order decisions like joining two large tables using an unselective join predicate. Another important reason is that in main memory picking an index-nested-loop join where a hash join would have been faster is never disastrous. With all data and indexes fully cached, we measured that the performance advantage of a hash join over an index-nested-loop join is at most $5\times$ with PostgreSQL and $2\times$ with HyPer. Obviously, when the index must be read from disk, random IO may result in a much larger factor. Therefore, the main-memory setting is much more forgiving.

4.3 Complex Access Paths

So far, all query executions were performed on a database with indexes on primary key attributes only. To see if the query optimization problem becomes harder when there are more indexes, we additionally indexed all foreign key attributes. Figure 7b shows the effect of additional foreign key indexes. We see large performance differences with 40% of the queries being slower by a factor of 2! Note that these results do not mean that adding more indexes decreases performance (although this can occasionally happen). Indeed overall performance generally increases significantly, but the more indexes are available the harder the job of the query optimizer becomes.

4.4 Join-Crossing Correlations

There is consensus in our community that estimation of intermediate result cardinalities in the presence of *correlated* query predicates is a frontier in query optimization research. The JOB workload studied in this paper consists of real-world data and its queries contain many correlated predicates. Our experiments that focus on *single-table* subquery cardinality estimation quality (cf. Table 1) show that systems that keep table samples (HyPer and presumably DBMS A) can achieve almost perfect estimation results, even for correlated predicates (inside the same table). As such, the cardinality estimation research challenge appears to lie in queries where the correlated predicates involve columns from *different* tables, connected by joins. These we call "join-crossing correlations". Such correlations frequently occur in the IMDB data set, e.g., actors born in Paris are likely to play in French movies.

Given these join-crossing correlations one could wonder if there exist complex access paths that allow to exploit these. One example relevant here despite its original setting in XQuery processing is ROX [22]. It studied runtime join order query optimization in the context of DBLP co-authorship queries that count how many `Authors` had published `Papers` in three particular venues, out of many. These queries joining the author sets from different venues clearly have join-crossing correlations, since authors who publish in `VLDB` are typically database researchers, likely to also publish in `SIGMOD`, but not—say—in `Nature`.

In the DBLP case, `Authorship` is a $n : m$ relationship that links the relation `Authors` with the relation `Papers`. The optimal query plans in [22] used an index-nested-loop join, looking up each `author` into `Authorship.author` (the indexed primary key) followed by a filter restriction on `Paper.venue`, which needs to be looked up with yet another join. This filter on venue would normally have to be calculated *after* these two joins. However, the physical design of [22] stored `Authorship` *partitioned by* `Paper.venue`.[7] This partitioning has startling effects: instead of one `Authorship` table and primary key index, one physically has many, one for each `venue` partition. This means that by accessing the right partition, the filter is implicitly enforced (for free), *before* the join happens. This specific physical design therefore causes the optimal plan to be as follows: first join the smallish authorship set from `SIGMOD` with the large set for `Nature` producing almost no result tuples, making the subsequent nested-loops index lookup join into `VLDB` very cheap. If the tables would not have been partitioned, index lookups from all `SIGMOD` authors into `Authorships` would first find all co-authored papers, of which the great majority is irrelevant because they are about database research, and were not published in `Nature`. Without this partitioning, there is no way to avoid this large intermediate result, and there is no query plan that comes close to the partitioned case in efficiency: even if cardinality estimation would be able to predict join-crossing correlations, there would be no physical way to profit from this knowledge.

The lesson to draw from this example is that the effects of query optimization are always gated by the available options in terms of access paths. Having a partitioned index on a *join-crossing predicate* as in [22] is a non-obvious physical design alternative which even modifies the schema by bringing in a join-crossing column (`Paper.venue`) as partitioning key of a table (`Authorship`). The partitioned DBLP set-up is just one example of how one particular join-crossing correlation can be handled, rather than a generic solution. Join-crossing correlations remain an open frontier for database research involving the interplay of physical design, query execution and query optimization. In our JOB experiments we do not attempt to chart this mostly unknown space, but rather characterize the impact of (join-crossing) correlations on the current state-of-the-art of query processing, restricting ourselves to standard PK and FK indexing.

5. COST MODELS

The cost model guides the selection of plans from the search space. The cost models of contemporary systems are sophisticated

[7] In fact, rather than relational table partitioning, there was a separate XML document per venue, e.g., separate documents for `SIGMOD`, `VLDB`, `Nature` and a few thousand more venues. Storage in a separate XML document has roughly the same effect on access paths as partitioned tables.

software artifacts that are resulting from 30+ years of research and development, mostly concentrated in the area of traditional disk-based systems. PostgreSQL's cost model, for instance, is comprised of over 4000 lines of C code, and takes into account various subtle considerations, e.g., it takes into account partially correlated index accesses, interesting orders, tuple sizes, etc. It is interesting, therefore, to evaluate how much a complex cost model actually contributes to the overall query performance.

First, we will experimentally establish the correlation between the PostgreSQL cost model—a typical cost model of a disk-based DBMS—and the query runtime. Then, we will compare the PostgreSQL cost model with two other cost functions. The first cost model is a tuned version of PostgreSQL's model for a main-memory setup where all data fits into RAM. The second cost model is an extremely simple function that only takes the number of tuples produced during query evaluation into account. We show that, unsurprisingly, the difference between the cost models is dwarfed by the cardinality estimates errors. We conduct our experiments on a database instance with foreign key indexes. We begin with a brief description of a typical disk-oriented complex cost model, namely the one of PostgreSQL.

5.1 The PostgreSQL Cost Model

PostgreSQL's disk-oriented cost model combines CPU and I/O costs with certain weights. Specifically, the cost of an operator is defined as a weighted sum of the number of accessed disk pages (both sequential and random) and the amount of data processed in memory. The cost of a query plan is then the sum of the costs of all operators. The default values of the weight parameters used in the sum (*cost variables*) are set by the optimizer designers and are meant to reflect the relative difference between random access, sequential access and CPU costs.

The PostgreSQL documentation contains the following note on cost variables: *"Unfortunately, there is no well-defined method for determining ideal values for the cost variables. They are best treated as averages over the entire mix of queries that a particular installation will receive. This means that changing them on the basis of just a few experiments is very risky."* For a database administrator, who needs to actually set these parameters these suggestions are not very helpful; no doubt most will not change these parameters. This comment is of course, not PostgreSQL-specific, since other systems feature similarly complex cost models. In general, tuning and calibrating cost models (based on sampling, various machine learning techniques etc.) has been a subject of a number of papers (e.g, [42, 25]). It is important, therefore, to investigate the impact of the cost model on the overall query engine performance. This will indirectly show the contribution of cost model errors on query performance.

5.2 Cost and Runtime

The main virtue of a cost function is its ability to predict which of the alternative query plans will be the fastest, given the cardinality estimates; in other words, what counts is its correlation with the query runtime. The correlation between the cost and the runtime of queries in PostgreSQL is shown in Figure 8a. Additionally, we consider the case where the engine has the true cardinalities injected, and plot the corresponding data points in Figure 8b. For both plots, we fit the linear regression model (displayed as a straight line) and highlight the standard error. The predicted cost of a query correlates with its runtime in both scenarios. Poor cardinality estimates, however, lead to a large number of outliers and a very wide standard error area in Figure 8a. Only using the true cardinalities makes

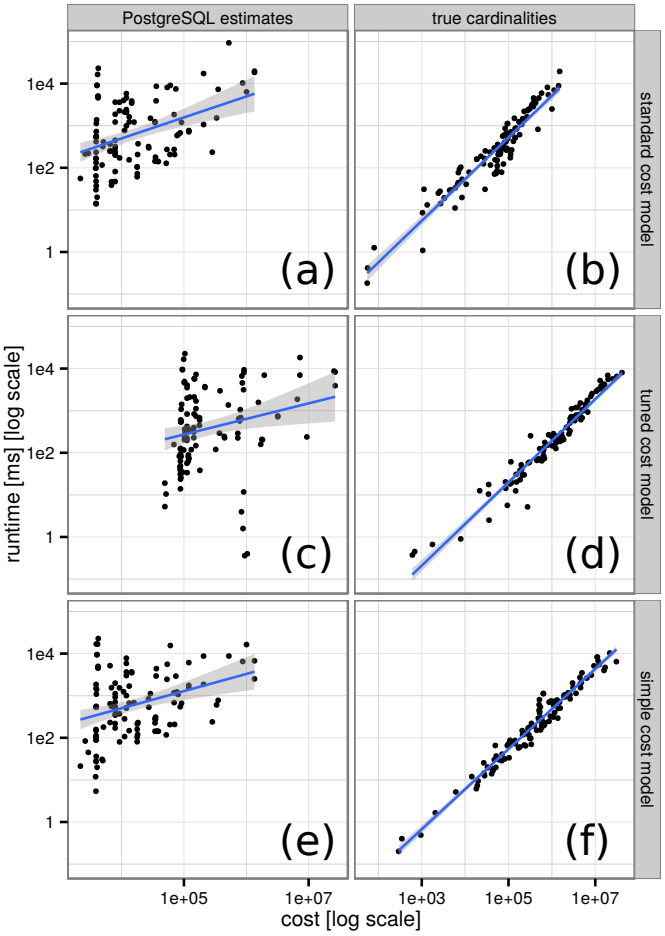


Figure 8: Predicted cost vs. runtime for different cost models

the PostgreSQL cost model a reliable predictor of the runtime, as has been observed previously [42].

Intuitively, a straight line in Figure 8 corresponds to an ideal cost model that always assigns (predicts) higher costs for more expensive queries. Naturally, any monotonically increasing function would satisfy that requirement, but the linear model provides the simplest and the closest fit to the observed data. We can therefore interpret the deviation from this line as the *prediction error* of the cost model. Specifically, we consider the absolute percentage error of a cost model for a query Q: $\epsilon(Q) = \frac{|T_{\text{real}}(Q) - T_{\text{pred}}(Q)|}{T_{\text{real}}(Q)}$, where T_{real} is the observed runtime, and T_{pred} is the runtime predicted by our linear model. Using the default cost model of PostgreSQL and the true cardinalities, the median error of the cost model is 38%.

5.3 Tuning the Cost Model for Main Memory

As mentioned above, a cost model typically involves parameters that are subject to tuning by the database administrator. In a disk-based system such as PostgreSQL, these parameters can be grouped into *CPU cost parameters* and *I/O cost parameters*, with the default settings reflecting an expected proportion between these two classes in a hypothetical workload.

In many settings the default values are sub optimal. For example, the default parameter values in PostgreSQL suggest that processing a tuple is 400x cheaper than reading it from a page. However, modern servers are frequently equipped with very large RAM capacities, and in many workloads the data set actually fits entirely

into available memory (admittedly, the core of PostgreSQL was shaped decades ago when database servers only had few megabytes of RAM). This does not eliminate the page access costs entirely (due to buffer manager overhead), but significantly bridges the gap between the I/O and CPU processing costs.

Arguably, the most important change that needs to be done in the cost model for a main-memory workload is to decrease the proportion between these two groups. We have done so by multiplying the *CPU cost* parameters by a factor of 50. The results of the workload run with improved parameters are plotted in the two middle subfigures of Figure 8. Comparing Figure 8b with d, we see that tuning does indeed improve the correlation between the cost and the runtime. On the other hand, as is evident from comparing Figure 8c and d, parameter tuning improvement is still overshadowed by the difference between the estimated and the true cardinalities. Note that Figure 8c features a set of outliers for which the optimizer has accidentally discovered very good plans (runtimes around 1 ms) without realizing it (hence very high costs). This is another sign of "oscillation" in query planning caused by cardinality misestimates.

In addition, we measure the prediction error ϵ of the tuned cost model, as defined in Section 5.2. We observe that tuning improves the predictive power of the cost model: the median error decreases from 38% to 30%.

5.4 Are Complex Cost Models Necessary?

As discussed above, the PostgreSQL cost model is quite complex. Presumably, this complexity should reflect various factors influencing query execution, such as the speed of a disk seek and read, CPU processing costs, etc. In order to find out whether this complexity is actually necessary in a main-memory setting, we will contrast it with a very simple cost function C_{mm}. This cost function is tailored for the *main-memory* setting in that it does not model I/O costs, but only counts the number of tuples that pass through each operator during query execution:

$$C_{\text{mm}}(T) = \begin{cases} \tau \cdot |R| & \text{if } T = R \vee T = \sigma(R) \\ |T| + C_{\text{mm}}(T_1) + C_{\text{mm}}(T_2) & \text{if } T = T_1 \bowtie^{\text{HJ}} T_2 \\ C_{\text{mm}}(T_1) + & \text{if } T = T_1 \bowtie^{\text{INL}} T_2, \\ \quad \lambda \cdot |T_1| \cdot \max(\frac{|T_1 \bowtie R|}{|T_1|}, 1) & (T_2 = R \vee T_2 = \sigma(R)) \end{cases}$$

In the formula above R is a base relation, and $\tau \leq 1$ is a parameter that discounts the cost of a table scan in comparison with joins. The cost function distinguishes between hash $\bowtie^{\text{HJ}}$ and index-nested loop $\bowtie^{\text{INL}}$ joins: the latter scans T_1 and performs index lookups into an index on R, thus avoiding a full table scan of R. A special case occurs when there is a selection on the right side of the index-nested loop join, in which case we take into account the number of tuple lookups in the base table index and essentially discard the selection from the cost computation (hence the multiplier $\max(\frac{|T_1 \bowtie R|}{|T_1|}, 1)$). For index-nested loop joins we use the constant $\lambda \geq 1$ to approximate by how much an index lookup is more expensive than a hash table lookup. Specifically, we set $\lambda = 2$ and $\tau = 0.2$. As in our previous experiments, we disable nested loop joins when the inner relation is not an index lookup (i.e., non-index nested loop joins).

The results of our workload run with C_{mm} as a cost function are depicted in Figure 8e and f. We see that even our trivial cost model is able to fairly accurately predict the query runtime using the true cardinalities. To quantify this argument, we measure the improvement in the runtime achieved by changing the cost model for true cardinalities: In terms of the geometric mean over all queries, our tuned cost model yields 41% faster runtimes than the standard PostgreSQL model, but even a simple C_{mm} makes queries 34% faster

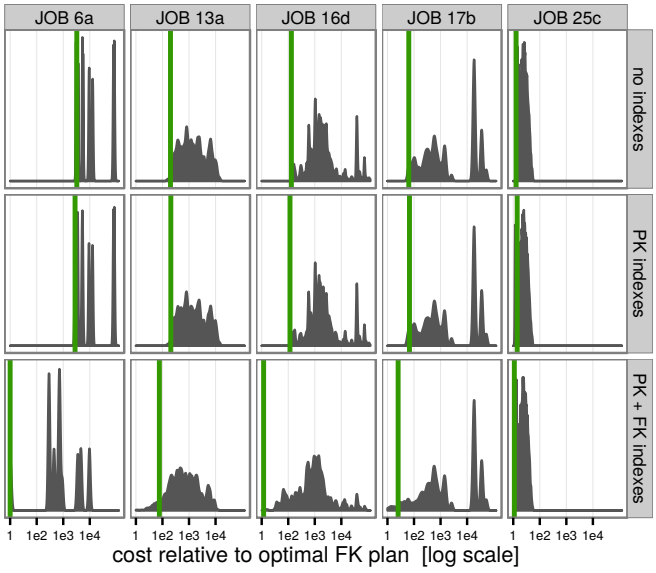


Figure 9: Cost distributions for 5 queries and different index configurations. The vertical green lines represent the cost of the optimal plan

than the built-in cost function. This improvement is not insignificant, but on the other hand, it is dwarfed by improvement in query runtime observed when we replace estimated cardinalities with the real ones (cf. Figure 6b). This allows us to reiterate our main message that cardinality estimation is much more crucial than the cost model.

6. PLAN SPACE

Besides cardinality estimation and the cost model, the final important query optimization component is a plan enumeration algorithm that explores the space of semantically equivalent join orders. Many different algorithms, both exhaustive (e.g., [29, 12]) as well as heuristic (e.g, [37, 32]) have been proposed. These algorithms consider a different number of candidate solutions (that constitute the *search space*) when picking the best plan. In this section we investigate how large the search space needs to be in order to find a good plan.

The experiments of this section use a standalone query optimizer, which implements Dynamic Programming (DP) and a number of heuristic join enumeration algorithms. Our optimizer allows the injection of arbitrary cardinality estimates. In order to fully explore the search space, we do not actually execute the query plans produced by the optimizer in this section, as that would be infeasible due to the number of joins our queries have. Instead, we first run the query optimizer using the estimates as input. Then, we recompute the cost of the resulting plan with the true cardinalities, giving us a very good approximation of the runtime the plan would have in reality. We use the in-memory cost model from Section 5.4 and assume that it perfectly predicts the query runtime, which, for our purposes, is a reasonable assumption since the errors of the cost model are negligible in comparison the cardinality errors. This approach allows us to compare a large number of plans without executing all of them.

6.1 How Important Is the Join Order?

We use the Quickpick [40] algorithm to visualize the costs of different join orders. Quickpick is a simple, randomized algorithm

that picks joins edges at random until all joined relations are fully connected. Each run produces a correct, but usually slow, query plan. By running the algorithm 10,000 times per query and computing the costs of the resulting plans, we obtain an approximate distribution for the costs of random plans. Figure 9 shows density plots for 5 representative example queries and for three physical database designs: no indexes, primary key indexes only, and primary+foreign key indexes. The costs are normalized by the optimal plan (with foreign key indexes), which we obtained by running dynamic programming and the true cardinalities.

The graphs, which use a logarithmic scale on the horizontal cost axis, clearly illustrate the importance of the join ordering problem: The slowest or even median cost is generally multiple orders of magnitude more expensive than the cheapest plan. The shapes of the distributions are quite diverse. For some queries, there are many good plans (e.g., 25c), for others few (e.g., 16d). The distribution are sometimes wide (e.g., 16d) and sometimes narrow (e.g., 25c). The plots for the "no indexes" and the "PK indexes" configurations are very similar implying that for our workload primary key indexes alone do not improve performance very much, since we do not have selections on primary key columns. In many cases the "PK+FK indexes" distributions have additional small peaks on the left side of the plot, which means that the optimal plan in this index configuration is much faster than in the other configurations.

We also analyzed the entire workload to confirm these visual observations: The percentage of plans that are at most $1.5\times$ more expensive than the optimal plan is 44% without indexes, 39% with primary key indexes, but only 4% with foreign key indexes. The average fraction between the worst and the best plan, i.e., the width of the distribution, is $101\times$ without indexes, $115\times$ with primary key indexes, and $48120\times$ with foreign key indexes. These summary statistics highlight the dramatically different search spaces of the three index configurations.

6.2 Are Bushy Trees Necessary?

Most join ordering algorithms do not enumerate all possible tree shapes. Virtually all optimizers ignore join orders with cross products, which results in a dramatically reduced optimization time with only negligible query performance impact. Oracle goes even further by not considering bushy join trees [1]. In order to quantify the effect of restricting the search space on query performance, we modified our DP algorithm to only enumerate *left-deep*, *right-deep*, or *zig-zag* trees.

Aside from the obvious tree shape restriction, each of these classes implies constraints on the join method selection. We follow the definition by Garcia-Molina et al.'s textbook, which is reverse from the one in Ramakrishnan and Gehrke's book: Using hash joins, right-deep trees are executed by first creating hash tables out of each relation except one before probing in all of these hash tables in a pipelined fashion, whereas in left-deep trees, a new hash table is built from the result of each join. In zig-zag trees, which are a super set of all left- and right-deep trees, each join operator must have at least one base relation as input. For index-nested loop joins we additionally employ the following convention: the left child of a join is a source of tuples that are looked up in the index on the right child, which must be a base table.

Using the true cardinalities, we compute the cost of the optimal plan for each of the three restricted tree shapes. We divide these costs by the optimal tree (which may have any shape, including "bushy") thereby measuring how much performance is lost by restricting the search space. The results in Table 2 show that zig-zag trees offer decent performance in most cases, with the worst case being $2.54\times$ more expensive than the best bushy plan. Left-deep trees are worse than zig-zag trees, as expected, but still result in reasonable performance. Right-deep trees, on the other hand, perform much worse than the other tree shapes and thus should not be used exclusively. The bad performance of right-deep trees is caused by the large intermediate hash tables that need to be created from each base relation and the fact that only the bottom-most join can be done via index lookup.

	PK indexes			PK + FK indexes		
	median	95%	max	median	95%	max
zig-zag	1.00	1.06	1.33	1.00	1.60	2.54
left-deep	1.00	1.14	1.63	1.06	2.49	4.50
right-deep	1.87	4.97	6.80	47.2	30931	738349

Table 2: Slowdown for restricted tree shapes in comparison to the optimal plan (true cardinalities)

6.3 Are Heuristics Good Enough?

So far in this paper, we have used the dynamic programming algorithm, which computes the optimal join order. However, given the bad quality of the cardinality estimates, one may reasonably ask whether an exhaustive algorithm is even necessary. We therefore compare dynamic programming with a randomized and a greedy heuristics.

The "Quickpick-1000" heuristics is a randomized algorithm that chooses the cheapest (based on the estimated cardinalities) 1000 random plans. Among all greedy heuristics, we pick Greedy Operator Ordering (GOO) since it was shown to be superior to other deterministic approximate algorithms [11]. GOO maintains a set of join trees, each of which initially consists of one base relation. The algorithm then combines the pair of join trees with the lowest cost to a single join tree. Both Quickpick-1000 and GOO can produce bushy plans, but obviously only explore parts of the search space. All algorithms in this experiment internally use the PostgreSQL cardinality estimates to compute a query plan, for which we compute the "true" cost using the true cardinalities.

Table 3 shows that it is worthwhile to fully examine the search space using dynamic programming despite cardinality misestimation. However, the errors introduced by estimation errors cause larger performance losses than the heuristics. In contrast to some other heuristics (e.g., [5]), GOO and Quickpick-1000 are not really aware of indexes. Therefore, GOO and Quickpick-1000 work better when few indexes are available, which is also the case when there are more good plans.

To summarize, our results indicate that enumerating all bushy trees exhaustively offers moderate but not insignificant performance benefits in comparison with algorithms that enumerate only a sub set of the search space. The performance potential from good cardinality estimates is certainly much larger. However, given the existence of exhaustive enumeration algorithms that can find the optimal solution for queries with dozens of relations very quickly (e.g., [29, 12]), there are few cases where resorting to heuristics or disabling bushy trees should be necessary.

7. RELATED WORK

Our cardinality estimation experiments show that systems which keep table samples for cardinality estimation predict single-table result sizes considerably better than those which apply the independence assumption and use single-column histograms [20]. We think systems should be adopting table samples as a simple and robust technique, rather than earlier suggestions to explicitly detect

	PK indexes						PK + FK indexes					
	PostgreSQL estimates			true cardinalities			PostgreSQL estimates			true cardinalities		
	median	95%	max	median	95%	max	median	95%	max	median	95%	max
Dynamic Programming	1.03	1.85	4.79	1.00	1.00	1.00	1.66	169	186367	1.00	1.00	1.00
Quickpick-1000	1.05	2.19	7.29	1.00	1.07	1.14	2.52	365	186367	1.02	4.72	32.3
Greedy Operator Ordering	1.19	2.29	2.36	1.19	1.64	1.97	2.35	169	186367	1.20	5.77	21.0

Table 3: Comparison of exhaustive dynamic programming with the Quickpick-1000 (best of 1000 random plans) and the Greedy Operator Ordering heuristics. All costs are normalized by the optimal plan of that index configuration

certain correlations [19] to subsequently create multi-column histograms [34] for these.

However, many of our JOB queries contain join-crossing correlations, which single-table samples do not capture, and where the current generation of systems still apply the independence assumption. There is a body of existing research work to better estimate result sizes of queries with join-crossing correlations, mainly based on join samples [17], possibly enhanced against skew (end-biased sampling [10], correlated samples [43]), using sketches [35] or graphical models [39]. This work confirms that without addressing join-crossing correlations, cardinality estimates deteriorate strongly with more joins [21], leading to both the over- and underestimation of result sizes (mostly the latter), so it would be positive if some of these techniques would be adopted by systems.

Another way of learning about join-crossing correlations is by exploiting query feedback, as in the LEO project [38], though there it was noted that deriving cardinality estimations based on a mix of exact knowledge and lack of knowledge needs a sound mathematical underpinning. For this, maximum entropy (MaxEnt [28, 23]) was defined, though the costs for applying maximum entropy are high and have prevented its use in systems so far. We found that the performance impact of estimation mistakes heavily depends on the physical database design; in our experiments the largest impact is in situations with the richest designs. From the ROX [22] discussion in Section 4.4 one might conjecture that to truly unlock the potential of correctly predicting cardinalities for join-crossing correlations, we also need new physical designs and access paths.

Another finding in this paper is that the adverse effects of cardinality misestimations can be strongly reduced if systems would be "hedging their bets" and not only choose the plan with the cheapest expected cost, but take the probabilistic distribution of the estimate into account, to avoid plans that are marginally faster than others but bear a high risk of strong underestimation. There has been work both on doing this for cardinality estimates purely [30], as well as combining these with a cost model (cost distributions [2]).

The problem with fixed hash table sizes for PostgreSQL illustrates that cost misestimation can often be mitigated by making the runtime behavior of the query engine more "performance robust". This links to a body of work to make systems adaptive to estimation mistakes, e.g., dynamically switch sides in a join, or change between hashing and sorting (GJoin [15]), switch between sequential scan and index lookup (smooth scan [4]), adaptively reordering join pipelines during query execution [24], or change aggregation strategies at runtime depending on the actual number of group-by values [31] or partition-by values [3].

A radical approach is to move query optimization to runtime, when actual value-distributions become available [33, 9]. However, runtime techniques typically restrict the plan search space to limit runtime plan exploration cost, and sometimes come with functional restrictions such as to only consider (sampling through) operators which have pre-created indexed access paths (e.g., ROX [22]).

Our experiments with the second query optimizer component besides cardinality estimation, namely the cost model, suggest that tuning cost models provides less benefits than improving cardinality estimates, and in a main-memory setting even an extremely simple cost-model can produce satisfactory results. This conclusion resonates with some of the findings in [42] which sets out to improve cost models but shows major improvements by refining cardinality estimates with additional sampling.

For testing the final query optimizer component, plan enumeration, we borrowed in our methodology from the Quickpick method used in randomized query optimization [40] to characterize and visualize the search space. Another well-known search space visualization method is Picasso [18], which visualizes query plans as areas in a space where query parameters are the dimensions. Interestingly, [40] claims in its characterization of the search space that good query plans are easily found, but our tests indicate that the richer the physical design and access path choices, the rarer good query plans become.

Query optimization is a core database research topic with a huge body of related work, that cannot be fully represented in this section. After decades of work still having this problem far from resolved [26], some have even questioned it and argued for the need of optimizer application hints [6]. This paper introduces the Join Order Benchmark based on the highly correlated IMDB real-world data set and a methodology for measuring the accuracy of cardinality estimation. Its integration in systems proposed for testing and evaluating the quality of query optimizers [41, 16, 14, 27] is hoped to spur further innovation in this important topic.

8. CONCLUSIONS AND FUTURE WORK

In this paper we have provided quantitative evidence for conventional wisdom that has been accumulated in three decades of practical experience with query optimizers. We have shown that query optimization is essential for efficient query processing and that exhaustive enumeration algorithms find better plans than heuristics. We have also shown that relational database systems produce large estimation errors that quickly grow as the number of joins increases, and that these errors are usually the reason for bad plans. In contrast to cardinality estimation, the contribution of the cost model to the overall query performance is limited.

Going forward, we see two main routes for improving the plan quality in heavily-indexed settings. First, database systems can incorporate more advanced estimation algorithms that have been proposed in the literature. The second route would be to increase the interaction between the runtime and the query optimizer. We leave the evaluation of both approaches for future work.

We encourage the community to use the Join Order Benchmark as a test bed for further experiments, for example into the risk/reward tradeoffs of complex access paths. Furthermore, it would be interesting to investigate disk-resident and distributed databases, which provide different challenges than our main-memory setting.

Acknowledgments

We would like to thank Guy Lohman and the anonymous reviewers for their valuable feedback. We also thank Moritz Wilfer for his input in the early stages of this project.

9. REFERENCES

[1] R. Ahmed, R. Sen, M. Poess, and S. Chakkappen. Of snowstorms and bushy trees. *PVLDB*, 7(13):1452–1461, 2014.

[2] B. Babcock and S. Chaudhuri. Towards a robust query optimizer: A principled and practical approach. In *SIGMOD*, pages 119–130, 2005.

[3] S. Bellamkonda, H.-G. Li, U. Jagtap, Y. Zhu, V. Liang, and T. Cruanes. Adaptive and big data scale parallel execution in Oracle. *PVLDB*, 6(11):1102–1113, 2013.

[4] R. Borovica-Gajic, S. Idreos, A. Ailamaki, M. Zukowski, and C. Fraser. Smooth scan: Statistics-oblivious access paths. In *ICDE*, pages 315–326, 2015.

[5] N. Bruno, C. A. Galindo-Legaria, and M. Joshi. Polynomial heuristics for query optimization. In *ICDE*, pages 589–600, 2010.

[6] S. Chaudhuri. Query optimizers: time to rethink the contract? In *SIGMOD*, pages 961–968, 2009.

[7] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. Exact cardinality query optimization for optimizer testing. *PVLDB*, 2(1):994–1005, 2009.

[8] M. Colgan. Oracle adaptive joins. `https://blogs.oracle.com/optimizer/entry/what_s_new_in_12c`, 2013.

[9] A. Dutt and J. R. Haritsa. Plan bouquets: query processing without selectivity estimation. In *SIGMOD*, pages 1039–1050, 2014.

[10] C. Estan and J. F. Naughton. End-biased samples for join cardinality estimation. In *ICDE*, page 20, 2006.

[11] L. Fegaras. A new heuristic for optimizing large queries. In *DEXA*, pages 726–735, 1998.

[12] P. Fender and G. Moerkotte. Counter strike: Generic top-down join enumeration for hypergraphs. *PVLDB*, 6(14):1822–1833, 2013.

[13] P. Fender, G. Moerkotte, T. Neumann, and V. Leis. Effective and robust pruning for top-down join enumeration algorithms. In *ICDE*, pages 414–425, 2012.

[14] C. Fraser, L. Giakoumakis, V. Hamine, and K. F. Moore-Smith. Testing cardinality estimation models in SQL Server. In *DBtest*, 2012.

[15] G. Graefe. A generalized join algorithm. In *BTW*, pages 267–286, 2011.

[16] Z. Gu, M. A. Soliman, and F. M. Waas. Testing the accuracy of query optimizers. In *DBTest*, 2012.

[17] P. J. Haas, J. F. Naughton, S. Seshadri, and A. N. Swami. Selectivity and cost estimation for joins based on random sampling. *J Computer System Science*, 52(3):550–569, 1996.

[18] J. R. Haritsa. The Picasso database query optimizer visualizer. *PVLDB*, 3(2):1517–1520, 2010.

[19] I. F. Ilyas, V. Markl, P. J. Haas, P. Brown, and A. Aboulnaga. CORDS: automatic discovery of correlations and soft functional dependencies. In *SIGMOD*, pages 647–658, 2004.

[20] Y. E. Ioannidis. The history of histograms (abridged). In *VLDB*, pages 19–30, 2003.

[21] Y. E. Ioannidis and S. Christodoulakis. On the propagation of errors in the size of join results. In *SIGMOD*, 1991.

[22] R. A. Kader, P. A. Boncz, S. Manegold, and M. van Keulen. ROX: run-time optimization of XQueries. In *SIGMOD*, pages 615–626, 2009.

[23] R. Kaushik, C. Ré, and D. Suciu. General database statistics using entropy maximization. In *DBPL*, pages 84–99, 2009.

[24] Q. Li, M. Shao, V. Markl, K. S. Beyer, L. S. Colby, and G. M. Lohman. Adaptively reordering joins during query execution. In *ICDE*, pages 26–35, 2007.

[25] F. Liu and S. Blanas. Forecasting the cost of processing multi-join queries via hashing for main-memory databases. In *SoCC*, pages 153–166, 2015.

[26] G. Lohman. Is query optimization a solved problem? `http://wp.sigmod.org/?p=1075`, 2014.

[27] L. F. Mackert and G. M. Lohman. R* optimizer validation and performance evaluation for local queries. In *SIGMOD*, pages 84–95, 1986.

[28] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. J. Haas, and U. Srivastava. Consistently estimating the selectivity of conjuncts of predicates. In *VLDB*, pages 373–384, 2005.

[29] G. Moerkotte and T. Neumann. Dynamic programming strikes back. In *SIGMOD*, pages 539–552, 2008.

[30] G. Moerkotte, T. Neumann, and G. Steidl. Preventing bad plans by bounding the impact of cardinality estimation errors. *PVLDB*, 2(1):982–993, 2009.

[31] I. Müller, P. Sanders, A. Lacurie, W. Lehner, and F. Färber. Cache-efficient aggregation: Hashing is sorting. In *SIGMOD*, pages 1123–1136, 2015.

[32] T. Neumann. Query simplification: graceful degradation for join-order optimization. In *SIGMOD*, pages 403–414, 2009.

[33] T. Neumann and C. A. Galindo-Legaria. Taking the edge off cardinality estimation errors using incremental execution. In *BTW*, pages 73–92, 2013.

[34] V. Poosala and Y. E. Ioannidis. Selectivity estimation without the attribute value independence assumption. In *VLDB*, pages 486–495, 1997.

[35] F. Rusu and A. Dobra. Sketches for size of join estimation. *TODS*, 33(3), 2008.

[36] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *SIGMOD*, pages 23–34, 1979.

[37] M. Steinbrunn, G. Moerkotte, and A. Kemper. Heuristic and randomized optimization for the join ordering problem. *VLDB J.*, 6(3):191–208, 1997.

[38] M. Stillger, G. M. Lohman, V. Markl, and M. Kandil. LEO - DB2's learning optimizer. In *VLDB*, pages 19–28, 2001.

[39] K. Tzoumas, A. Deshpande, and C. S. Jensen. Lightweight graphical models for selectivity estimation without independence assumptions. *PVLDB*, 4(11):852–863, 2011.

[40] F. Waas and A. Pellenkoft. Join order selection - good enough is easy. In *BNCOD*, pages 51–67, 2000.

[41] F. M. Waas, L. Giakoumakis, and S. Zhang. Plan space analysis: an early warning system to detect plan regressions in cost-based optimizers. In *DBTest*, 2011.

[42] W. Wu, Y. Chi, S. Zhu, J. Tatemura, H. Hacigümüs, and J. F. Naughton. Predicting query execution time: Are optimizer cost models really unusable? In *ICDE*, pages 1081–1092, 2013.

[43] F. Yu, W. Hou, C. Luo, D. Che, and M. Zhu. CS2: a new database synopsis for query estimation. In *SIGMOD*, pages 469–480, 2013.